

Nature-Inspired Genetic Algorithm for bin-packing problem variants

ECM3412 - Nature-Inspired Computation

Candidate Number: 710080994

University of Exeter
Exeter, United Kingdom

Abstract—We study the application of a genetic algorithm (GA) to a balanced bin-packing formulation. Implementing tournament selection, uniform crossover ($p_c = 0.8$), random reassignment mutation, and generational replacement with 1-elitism in the GA. Evaluating performance on two canonical instances: BPP1 ($k=500, b=10$, weight $w_i = i$) and BPP2 ($k=500, b=50$, weight $w_i = i^2/2$), terminating each trial after 10,000 fitness evaluations. A grid over mutation rate $p_m \in \{0.01, 0.05\}$ and tournament size $t \in \{3, 7\}$ is executed with five trials per setting. We report best-of-run fitness and discuss the effect of selection pressure and mutation on convergence and solution quality.

Index Terms—[genetic algorithms, bin-packing, optimization, nature-inspired computation]

I. INTRODUCTION

Bin-packing problem is a classic NP-hard optimisation problem in which k items must be assigned to b bins so as to minimise the difference d between the heaviest and the lightest bin.[3] Nature-inspired metaheuristics—notably genetic algorithms (GAs)—offer flexible search and optimization technique inspired by the process of natural selection and genetics in biological evolution. It's commonly used to find approximate or optimal solutions to complex problems like BPP.[1] This paper presents a clean GA baseline for the balanced bin-packing problem (BPP) and a small-scale experimental study of key parameters.

II. LITERATURE REVIEW

A. Genetic Algorithm Variants

The GA variants for bin-packing is a population-based evolutionary metaheuristic inspired by natural selection. This is a steady-state elitist GA emphasizing load balancing, not capacity minimization. It uses simple operators, avoids repair heuristics, and focuses on convergence stability and reproducibility across linear weights and quadratic weights.

B. Particle Swarm Optimization

In discrete combinatorial problems like BPP, continuous PSO must be discretized (binary or integer PSO), which can degrade solution accuracy and increase premature convergence risk.[2] Although PSO converges faster than GA because it continuously updates all candidate solutions using collective information, its poor exploration in large discrete spaces and it tends to cluster particles early which causes the loss of diversity.[2] So, PSO is not concided to implement in the GA.

C. Ant Colony Optimization

Although ACO has a high computational cost in updating pheromone matrices ($O(n^2)$) and sensitive to parameter tuning, it provides strong local exploitation and constructive search and having a good adaptability for sequential decision problems.[4] ACO would replace the GA population evolution loop with an ant solution-construction and pheromone-update process, while keeping the same fitness function $100/(1+D)$.

D. Other Nature-Inspired Approaches

Simulated Annealing(SA) is inspired by cooling process in Thermodynamics, it provides good local search and escape from local minima, but comes with slow convergence.[8]

Differential Evolution(DE) is inspired by evolutionary differential mutation of vectors, which works with real-valued encodings and provides strong global search.[7]

Cuckoo Search(CS) is inspired by brood parasitism behavior, which provides excellent diversification, but randomness may slow convergence.[5]

Artificial Bee Colony(ABC) is inspired by foraging behavior of honey bees, it provides a strong adaptability, but the parameter will be sensitive with slower exploitation phase.[6]

E. Critical Analysis

The GA variant is particularly well-suited for discrete, assignment-based problems like BPP because its operators directly manipulate integer-encoded assignments without conversion. Although PSO is computationally efficient and converges quickly, struggles with discrete adaptation like BPP comparing the approaches. ACO offers high-quality results when properly tuned but incurs heavier computational costs and slower convergence than other approaches. SA and ABC offer decent balance better than DE in exploration ability. CS provides excellent diversification more than others.

III. DESCRIPTION OF RESULTS

A. Experimental Setup

The genetic algorithm begins with an initial population of size 100, where each chromosome is an integer array of length 500, and each gene $g_i \in \{0, \dots, b-1\}$ represents the bin assigned to item i . The initial genes are generated uniformly at random within the range $[0, b-1]$. During crossover, a uniform crossover operator is applied with a probability $p_c = 0.8$ per

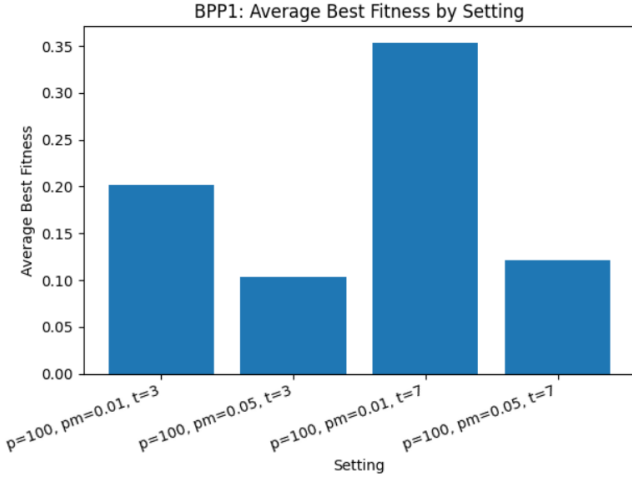
child, where a 0/1 mask determines which parent contributes each gene position. Mutation occurs on a per-gene basis with probability $pm \in \{0.01, 0.05\}$; any mutated gene is reassigned to a random bin within $[0, b - 1]$. The selection process uses tournament selection with a tournament size $t \in \{3, 7\}$, ensuring competitive pressure while maintaining diversity. The algorithm terminates after a hard cap of 10,000 fitness evaluations, including those from the initial population.

B. Experimental Results

1) BPP1 Results (10 bins, 500 items)

TABLE I
BPP1 RESULTS SUMMARY

p_m	Tournament Size	Best Fitness	Mean Fitness	Std Dev
0.01	3	0.278552	0.202014	0.056934
0.05	3	0.114811	0.103302	0.010977
0.01	7	0.396825	0.353397	0.041357
0.05	7	0.134409	0.121028	0.011280



2) BPP2 Results (50 bins, 500 items)

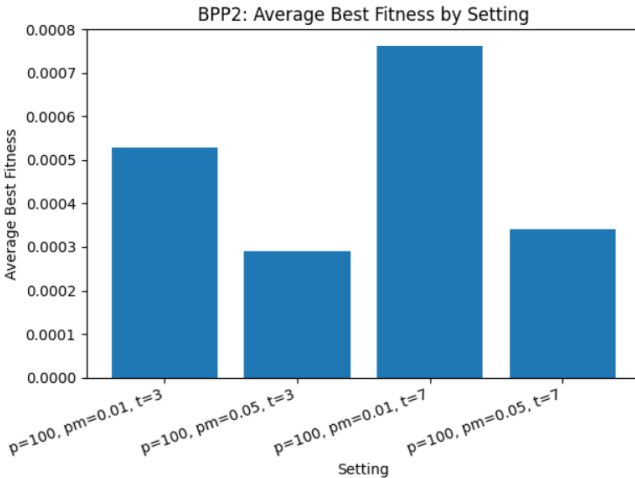


TABLE II
BPP2 RESULTS SUMMARY

p_m	Tournament Size	Best Fitness	Mean Fitness	Std Dev
0.01	3	0.000651	0.000528	0.000077
0.05	3	0.000299	0.000289	0.000011
0.01	7	0.000815	0.000762	0.000064
0.05	7	0.000389	0.000341	0.000032

IV. DISCUSSION AND FURTHER WORK

A. Question 1: Which combination of parameters produces the best results?

The combination of 0.01 mutation and tournament size of 7 produces the best results from both BPP1 and BPP2. It is because this combination tends to produce highest mean fitness, lowest variance across trials, and lower average D, comparing to other combination.

B. Question 2: What do you think is the reason for your findings in Question 1?

High selection pressure with high mutation has a very fast convergence, but risk of premature stagnation or solution volatility. Low selection pressure with high mutation has a high exploration, but slowly inconsistent progress and failing to concentrate on high-quality regions. Low selection pressure with low mutation is stable but slow convergence. High selection pressure with low mutation has fast convergence and stable improvement but moderate diversity, showing that the GA efficiently exploited promising regions without collapsing diversity too early.

C. Question 3: How do each of the parameter settings influence the performance of the algorithm?

Larger tournaments drives the population rapidly toward better fitness; smaller tournaments maintains greater diversity but slowing improvement. In BPP, tournament size of 7 achieves an effective trade-off, ensuring convergence toward well-balanced packings without getting trapped too early.

Although high mutation increases diversity and helps escape local minima, leading to oscillation in fitness. Low mutation produced higher average fitness and lower standard deviation, benefiting more from refinement than random exploration.

In BPP1, where bins are fewer and weights are linear, the search landscape is smooth and easily exploitable with many near-optimal configurations, so exploration beyond local optima is rarely necessary. In BPP2, with 50 bins and exponentially larger item weights, the landscape is more rugged and discontinuous due to quadratic weights, creating more local optima. Increasing b expands the solution space exponentially, this increases combinatorial complexity and makes the fitness landscape more irregular and rugged.

D. Question 4: Do you think that one of the algorithms in your literature review might have provided better results? Explain your answer.

ACO might provide better results in BPP2 than GA. ACO constructs solutions incrementally, using pheromone trails to

bias item–bin assignments based on prior success. So, it has strong ability to exploit structural regularities and gradually reinforce high-quality combinations.[4] its constructively feedback-driven nature could achieve better final balance once pheromone distributions stabilize.[4]

For BPP1, ACO might not outperform GA due to small problem size and smooth landscape, because it needs higher computational cost and comes with slower initial convergence.[4]

E. Further Experiments

Since the results of all experiments are sufficient to answer all the questions above, no additional experiments were performed for all trials and parameters.

F. Future Work

The GA is able to ensure clarity and reproducibility but limits efficiency. Although the algorithm consistently improves fitness, it may require many generations to reach highly balanced configurations.

Each experiment was constrained to 10,000 fitness evaluations to standardize comparisons. Larger populations completed fewer generations within the same evaluation limit, potentially understating their performance.

Future work should integrate problem-aware local repair into the GA. For example, applying a greedy load-balancing heuristic after crossover/mutation to reduce the bin-load difference. Or combining GA with a local search step that swaps items between the heaviest and lightest bins to refine elite individuals. Such hybrids typically achieve faster convergence and lower imbalance D without losing GA's exploratory capacity.

V. CONCLUSION

This provided a faithfully efficient Nature-Inspired genetic algorithm baseline for balanced bin packing and a small experimental study isolating the effects of mutation and selection pressure. The code is simple, fast, and reproducible, offering a strong foundation for further experimentation.

REFERENCES

- [1] [Reference 1 - nature-inspired approach for bin-packing]
- [2] [Reference 2 - Frankenstein's PSO: a composite particle swarm optimization algorithm]
- [3] [Reference 3 - bin-packing problem variants]
- [4] [Reference 4 - ACO algorithms for the traveling salesman problem]
- [5] [Reference 5 - Cuckoo search algorithm: a metaheuristic approach to solve structural optimization problems]
- [6] [Reference 6 - A global best artificial bee colony algorithm for global optimization]
- [7] [Reference 7 - A Review of Major Application Areas of Differential Evolution]
- [8] [Reference 8 - A new simulated annealing algorithm]